

Operating Systems Lab Assignment: Thread-Safe Data Structures

Cameron Cleveland

October 21, 2025

1 Introduction

This report documents the implementations and analyses for the thread-safe data structures lab assignment, covering thread-safe queue and stack implementations, Producer-Consumer and Undo-Redo use cases, and additional exercises.

2 Exercise 1: Thread-Safe Queue and Stack

```
#include <iostream>
#include <queue>
#include <stack>
#include <string>
#include <thread>
#include <vector>
#include <atomic>
#include <mutex>
#include <chrono>

// Thread-safe queue
template <typename T>
class ThreadSafeQueue {
private:
    std::queue<T> q;
    std::mutex mtx;
public:
    ThreadSafeQueue() = default;
    void push(const T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        q.push(value);
    }
    bool pop(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (q.empty()) return false;
        value = q.front();
        q.pop();
        return true;
    }
    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return q.empty();
    }
};
```

```

    }
    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return q.size();
    }
};

// Thread-safe stack
template <typename T>
class ThreadSafeStack {
private:
    std::stack<T> s;
    std::mutex mtx;
public:
    ThreadSafeStack() = default;
    void push(const T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        s.push(value);
    }
    bool pop(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (s.empty()) return false;
        value = s.top();
        s.pop();
        return true;
    }
    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return s.empty();
    }
    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return s.size();
    }
};

// Producer-consumer test
void producerConsumerTest() {
    std::cout << "Problem 1: Producer-Consumer Simulation\n";
    ThreadSafeQueue<std::string> mq;
    const int NUM_PRODUCERS = 3;
    const int NUM_CONSUMERS = 2;
    const int MESSAGES_PER_PRODUCER = 5;
    std::atomic<int> producedCount{0};

    auto producer = [&](int id) {
        for (int i = 0; i < MESSAGES_PER_PRODUCER; ++i) {
            std::string msg = "Producer " + std::to_string(id) + " -
                ↪ Message " + std::to_string(i);
            mq.push(msg);
            producedCount.fetch_add(1);
            std::this_thread::sleep_for(std::chrono::milliseconds(50));
        }
    };
};

```

```

auto consumer = [&](int id) {
    while (true) {
        std::string msg;
        if (mq.pop(msg)) {
            std::cout << "Consumer " << id << " processed: " << msg <<
                ↪ std::endl;
        } else {
            if (producedCount.load() >= NUM_PRODUCERS *
                ↪ MESSAGES_PER_PRODUCER && mq.empty()) break;
            std::this_thread::sleep_for(std::chrono::milliseconds(20))
                ↪ ;
        }
    }
};

std::vector<std::thread> producers, consumers;
for (int i = 0; i < NUM_PRODUCERS; ++i) producers.emplace_back(
    ↪ producer, i);
for (int i = 0; i < NUM_CONSUMERS; ++i) consumers.emplace_back(
    ↪ consumer, i);

for (auto &t : producers) t.join();
for (auto &t : consumers) t.join();
}

// Undo-redo test using thread-safe stacks
void undoRedoTest() {
    std::cout << "\nProblem 2: Undo-Redo System\n";
    ThreadSafeStack<std::string> undoStack;
    ThreadSafeStack<std::string> redoStack;
    std::string current = "Initial";

    auto edit = [&](const std::string& newText) {
        undoStack.push(current);
        current = newText;
        // clear redo stack (simple destructive loop)
        std::string tmp;
        while (redoStack.pop(tmp)) {}
        std::cout << "Edit: " << current << std::endl;
    };

    auto undo = [&]() {
        std::string prev;
        if (undoStack.pop(prev)) {
            redoStack.push(current);
            current = prev;
            std::cout << "Undo -> " << current << std::endl;
        } else {
            std::cout << "Undo stack empty\n";
        }
    };

    auto redo = [&]() {

```

```

        std::string next;
        if (redoStack.pop(next)) {
            undoStack.push(current);
            current = next;
            std::cout << "Redo -> " << current << std::endl;
        } else {
            std::cout << "Redo stack empty\n";
        }
    };

    edit("Hello");
    edit("Hello World");
    undo();
    undo();
    redo();
    edit("Hello Galaxy");
    undo();
}

int main() {
    producerConsumerTest();
    undoRedoTest();
    return 0;
}

```

Explanation: The queue and stack were made thread-safe by using a single mutex to lock each operation. Every function (push, pop, size, empty) is wrapped with a `std::lock_guard<std::mutex>` to ensure that only one thread can access or modify the structure at a time. In the Producer-Consumer simulation, multiple producers push messages while consumers continuously pop them until all messages are processed. The Undo-Redo system uses two stacks: edits push the previous state onto the undo stack and clear the redo stack, while undo and redo operations swap states safely between the two.

Analysis: This design ensures correctness and simplicity but limits concurrency because each operation holds the same lock. The termination logic checks both production count and emptiness, which works but could race under heavier loads. Using condition variables for blocking consumers would prevent busy-waiting and improve efficiency. Overall, the exercise highlights the balance between safety and performance in synchronization.

Screenshot: Include a screenshot of compiling and running `thread_safe_data_structures.cpp`.

3 Exercise 3: Thread-Safe Priority Queue

```

#include <iostream>
#include <queue>
#include <stack>
#include <string>
#include <thread>
#include <vector>
#include <atomic>
#include <mutex>
#include <chrono>

```

```

@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ touch thread_safe_data_structures.cpp
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ g++ -o thread_safe_data_structures thread_safe_data_structures.cpp -pthread -std=c++11
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ ./thread_safe_data_structures
Problem 1: Producer-Consumer Simulation
Consumer 0 processed: Producer 2 - Message 0
Consumer 0 processed: Producer 0 - Message 0
Consumer 0 processed: Producer 1 - Message 0
Consumer 0 processed: Producer 2 - Message 1
Consumer 1 processed: Producer 0 - Message 1
Consumer 1 processed: Producer 1 - Message 1
Consumer 1 processed: Producer 2 - Message 2
Consumer 1 processed: Producer 0 - Message 2
Consumer 0 processed: Producer 1 - Message 2
Consumer Consumer 10 processed: processed: Producer 0 - Message 3Producer 2 - Message 3
Consumer 0 processed: Producer 1 - Message 3

Consumer 1 processed: Producer 2 - Message 4
Consumer 1 processed: Producer 0 - Message 4
Consumer 1 processed: Producer 1 - Message 4

Problem 2: Undo-Redo System
Edit: Hello
Edit: Hello World
Undo → Hello
Undo → Initial
Redo → Hello
Edit: Hello Galaxy
Undo → Hello
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $

```

Figure 1: Compilation and execution of thread_safe_data_structures.cpp

```

// Thread-safe queue
template <typename T>
class ThreadSafeQueue {
private:
    std::queue<T> q;
    std::mutex mtx;
public:
    ThreadSafeQueue() = default;
    void push(const T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        q.push(value);
    }
    bool pop(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (q.empty()) return false;
        value = q.front();
        q.pop();
        return true;
    }
    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return q.empty();
    }
    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return q.size();
    }
};

// Thread-safe stack
template <typename T>
class ThreadSafeStack {
private:

```

```

    std::stack<T> s;
    std::mutex mtx;
public:
    ThreadSafeStack() = default;
    void push(const T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        s.push(value);
    }
    bool pop(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (s.empty()) return false;
        value = s.top();
        s.pop();
        return true;
    }
    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return s.empty();
    }
    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return s.size();
    }
};

// Producer-consumer test
void producerConsumerTest() {
    std::cout << "Problem 1: Producer-Consumer Simulation\n";
    ThreadSafeQueue<std::string> mq;
    const int NUM_PRODUCERS = 3;
    const int NUM_CONSUMERS = 2;
    const int MESSAGES_PER_PRODUCER = 5;
    std::atomic<int> producedCount{0};

    auto producer = [&](int id) {
        for (int i = 0; i < MESSAGES_PER_PRODUCER; ++i) {
            std::string msg = "Producer " + std::to_string(id) + " -
                ↪ Message " + std::to_string(i);
            mq.push(msg);
            producedCount.fetch_add(1);
            std::this_thread::sleep_for(std::chrono::milliseconds(50));
        }
    };

    auto consumer = [&](int id) {
        while (true) {
            std::string msg;
            if (mq.pop(msg)) {
                std::cout << "Consumer " << id << " processed: " << msg <<
                    ↪ std::endl;
            } else {
                if (producedCount.load() >= NUM_PRODUCERS *
                    ↪ MESSAGES_PER_PRODUCER && mq.empty()) break;
            }
        }
    };
}

```

```

        std::this_thread::sleep_for(std::chrono::milliseconds(20))
        ↪ ;
    }
}
};

std::vector<std::thread> producers, consumers;
for (int i = 0; i < NUM_PRODUCERS; ++i) producers.emplace_back(
    ↪ producer, i);
for (int i = 0; i < NUM_CONSUMERS; ++i) consumers.emplace_back(
    ↪ consumer, i);

for (auto &t : producers) t.join();
for (auto &t : consumers) t.join();
}

// Undo-redo test using thread-safe stacks
void undoRedoTest() {
    std::cout << "\nProblem 2: Undo-Redo System\n";
    ThreadSafeStack<std::string> undoStack;
    ThreadSafeStack<std::string> redoStack;
    std::string current = "Initial";

    auto edit = [&](const std::string& newText) {
        undoStack.push(current);
        current = newText;
        // clear redo stack (simple destructive loop)
        std::string tmp;
        while (redoStack.pop(tmp)) {}
        std::cout << "Edit: " << current << std::endl;
    };

    auto undo = [&]() {
        std::string prev;
        if (undoStack.pop(prev)) {
            redoStack.push(current);
            current = prev;
            std::cout << "Undo -> " << current << std::endl;
        } else {
            std::cout << "Undo stack empty\n";
        }
    };

    auto redo = [&]() {
        std::string next;
        if (redoStack.pop(next)) {
            undoStack.push(current);
            current = next;
            std::cout << "Redo -> " << current << std::endl;
        } else {
            std::cout << "Redo stack empty\n";
        }
    };
};

```

```

        edit("Hello");
        edit("Hello World");
        undo();
        undo();
        redo();
        edit("Hello Galaxy");
        undo();
    }

int main() {
    producerConsumerTest();
    undoRedoTest();
    return 0;
}

```

Explanation: The priority queue uses a `std::priority_queue` container wrapped with a mutex to protect insertion and removal. Each operation locks the queue before accessing the heap, maintaining order and preventing race conditions even when multiple threads push or pop simultaneously.

Analysis: The implementation is safe and correct but fully serialized, every push and pop operation competes for the same lock. For improved concurrency, condition variables could be used to block consumers until data becomes available. A more scalable solution might involve multiple internal queues or a lock-free heap, though this adds complexity beyond the lab scope.

Screenshot: Include a screenshot of compiling and running `priority_queue.cpp`.

```

@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ g++ -std=c++11 -o priority_queue priority_queue.cpp -pthread
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ ./priority_queue
bash: ./priority_queue: No such file or directory
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ ./priority_queue

Problem 1: Producer-Consumer Simulation
Consumer 0 processed: Producer 0 - Message 0
Consumer 0 processed: Producer 2 - Message 0
Consumer 1 processed: Producer 1 - Message 0
Consumer 0 processed: Producer 0 - Message 1
Consumer 0 processed: Producer 1 - Message 1
Consumer 1 processed: Producer 2 - Message 1
Consumer Consumer 01 processed: Producer 0 - Message 2 processed: Producer 1 - Message 2
Consumer 0 processed: Producer 2 - Message 2

Consumer 0 processed: Producer 0 - Message 3
Consumer 0 processed: Producer 2 - Message 3
Consumer 0 processed: Producer 1 - Message 3
Consumer 1 processed: Producer 1 - Message 4
Consumer 1 processed: Producer 0 - Message 4
Consumer 1 processed: Producer 2 - Message 4

Problem 2: Undo-Redo System
Edit: Hello
Edit: Hello World
Undo → Hello
Undo → Initial
Redo → Hello
Edit: Hello Galaxy
Undo → Hello
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $

```

Figure 2: Compilation and execution of `priority_queue.cpp`

4 Exercise 4: Thread-Safe Circular Buffer

```

#include <iostream>
#include <vector>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <chrono>

class ThreadSafeCircularBuffer {
private:
    std::vector<int> buf;
    size_t capacity;
    size_t head = 0;
    size_t tail = 0;
    size_t count = 0;
    std::mutex mtx;
    std::condition_variable not_full;
    std::condition_variable not_empty;
public:
    ThreadSafeCircularBuffer(size_t n = 8) : buf(n), capacity(n) {}
    void push(int v) {
        std::unique_lock<std::mutex> lock(mtx);
        not_full.wait(lock, [&]{ return count < capacity; });
        buf[tail] = v;
        tail = (tail + 1) % capacity;
        ++count;
        not_empty.notify_one();
    }
    bool pop(int &out) {
        std::unique_lock<std::mutex> lock(mtx);
        if (count == 0) return false;
        out = buf[head];
        head = (head + 1) % capacity;
        --count;
        not_full.notify_one();
        return true;
    }
    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return count == 0;
    }
    bool full() {
        std::lock_guard<std::mutex> lock(mtx);
        return count == capacity;
    }
    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return count;
    }
};

void circularBufferTest() {
    std::cout << "Exercise: Thread-Safe Circular Buffer\n";
    ThreadSafeCircularBuffer cb(6);

```

```

const int NUM_PRODUCERS = 2;
const int NUM_CONSUMERS = 2;
const int ITEMS = 12;

auto producer = [&](int id) {
    for (int i = 0; i < ITEMS/NUM_PRODUCERS; ++i) {
        int v = id * 100 + i;
        cb.push(v);
        std::cout << "Produced " << v << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(30));
    }
};

auto consumer = [&](int id) {
    for (;;) {
        int v;
        if (cb.pop(v)) {
            std::cout << "Consumed " << v << std::endl;
        } else {
            if (cb.empty()) break;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
};

std::vector<std::thread> producers, consumers;
for (int i = 0; i < NUM_PRODUCERS; ++i) producers.emplace_back(
    ↪ producer, i);
for (int i = 0; i < NUM_CONSUMERS; ++i) consumers.emplace_back(
    ↪ consumer, i);
for (auto &t : producers) t.join();
// give consumers time to drain
std::this_thread::sleep_for(std::chrono::milliseconds(500));
for (auto &t : consumers) t.join();
}

int main() {
    circularBufferTest();
    return 0;
}

```

Explanation: The circular buffer implementation uses a fixed-size vector with head, tail, and count indices. One mutex controls access, while two condition variables, `not_full` and `not_empty`, handle producer and consumer synchronization. Producers wait when the buffer is full, and consumers wait when it's empty, ensuring no overwriting or reading invalid data.

Analysis: The buffer introduces natural backpressure, keeping memory usage predictable and enforcing balance between production and consumption. The main challenges were avoiding off-by-one index errors and ensuring condition variables were properly signaled. Updating shared state within a locked region and notifying afterward kept the behavior consistent. This implementation shows how condition variables enable efficient coordination without wasting CPU cycles.

Screenshot: Include a screenshot of compiling and running `circular_buffer.cpp`.

```

@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ touch circular_buffer.cpp
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ g++ -o circular_buffer circular_buffer.cpp -pthread -std=c++11
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ ./circular_buffer
Exercise: Thread-Safe Circular Buffer
Produced 0
Produced 100
Consumed 100
Consumed 0
Produced 1
Produced 101
Consumed 1
Consumed 101
Produced 2
Produced 102
Produced 3
Produced 103
Consumed 2
Consumed 102
Produced 4
Produced 104
Consumed 3
Consumed 103
Produced 5
Produced 105
Consumed 4
Consumed 104
Consumed 5Consumed
105
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $

```

Figure 3: Compilation and execution of circular_buffer.cpp

5 Exercise 5: Thread-Safe Deque

```

#include <iostream>
#include <deque>
#include <thread>
#include <vector>
#include <mutex>
#include <chrono>

template <typename T>
class ThreadSafeDeque {
private:
    std::deque<T> dq;
    std::mutex mtx;
public:
    void push_front(const T& v) {
        std::lock_guard<std::mutex> lock(mtx);
        dq.push_front(v);
    }
    void push_back(const T& v) {
        std::lock_guard<std::mutex> lock(mtx);
        dq.push_back(v);
    }
    bool pop_front(T& out) {
        std::lock_guard<std::mutex> lock(mtx);
        if (dq.empty()) return false;
        out = dq.front();
        dq.pop_front();
        return true;
    }
    bool pop_back(T& out) {
        std::lock_guard<std::mutex> lock(mtx);
        if (dq.empty()) return false;

```

```

        out = dq.back();
        dq.pop_back();
        return true;
    }
    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return dq.empty();
    }
    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return dq.size();
    }
};

void dequeTest() {
    std::cout << "Exercise: Thread-Safe Deque\n";
    ThreadSafeDeque<int> dq;
    auto worker1 = [&]() {
        for (int i = 0; i < 10; ++i) { dq.push_back(i); std::this_thread::
            ↪ sleep_for(std::chrono::milliseconds(10)); }
    };
    auto worker2 = [&]() {
        for (int i = 100; i < 110; ++i) { dq.push_front(i); std::
            ↪ this_thread::sleep_for(std::chrono::milliseconds(15)); }
    };
    auto popper = [&]() {
        for (int i = 0; i < 10; ++i) {
            int v;
            if (dq.pop_front(v)) std::cout << "Pop front " << v << std::
                ↪ endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(20));
        }
    };

    std::thread t1(worker1), t2(worker2), t3(popper);
    t1.join(); t2.join(); t3.join();
}

int main() {
    dequeTest();
    return 0;
}

```

Explanation: The deque supports operations from both ends and is synchronized with a single mutex that guards all operations. Each push and pop action locks the deque to maintain data consistency and prevent race conditions during concurrent access.

Analysis: While simple and correct, this design can become a bottleneck when multiple threads push and pop from both ends simultaneously. A refined version could use separate locks for the head and tail to reduce contention but would need strict ordering to avoid deadlocks. This exercise demonstrates the trade-offs between fine-grained locking complexity and overall thread safety.

Screenshot: Include a screenshot of compiling and running `thread_safe_deque.cpp`.

```

@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ touch thread_safe_deque.cpp
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ g++ -o thread_safe_deque thread_safe_deque.cpp -pthread -std=c++11
@camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ ./thread_safe_deque
Exercise: Thread-Safe Deque
Pop front 100
Pop front 101
Pop front 102
Pop front 103
Pop front 105
Pop front 106
Pop front 107
Pop front 109
Pop front 108
Pop front 104

```

Figure 4: Compilation and execution of thread_safe_deque.cpp

6 Exercise 6: Thread-Safe Linked List

```

#include <iostream>
#include <thread>
#include <mutex>
#include <vector>

// A simple thread-safe singly linked list with coarse-grained locking
template <typename T>
class ThreadSafeLinkedList {
private:
    struct Node {
        T value;
        Node* next;
        Node(const T& v) : value(v), next(nullptr) {}
    };
    Node* head;
    std::mutex mtx;
public:
    ThreadSafeLinkedList() : head(nullptr) {}
    ~ThreadSafeLinkedList() {
        while (head) {
            Node* n = head;
            head = head->next;
            delete n;
        }
    }
    void push_front(const T& v) {
        std::lock_guard<std::mutex> lock(mtx);
        Node* n = new Node(v);
        n->next = head;
        head = n;
    }
    bool pop_front(T& out) {
        std::lock_guard<std::mutex> lock(mtx);
        if (!head) return false;
        Node* n = head;
        out = n->value;
        head = head->next;
        delete n;
        return true;
    }
}

```

```

    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return head == nullptr;
    }
    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        size_t c = 0;
        Node* cur = head;
        while (cur) { ++c; cur = cur->next; }
        return c;
    }
};

void linkedListTest() {
    std::cout << "Exercise: Thread-Safe Linked List\n";
    ThreadSafeLinkedList<int> list;
    auto pusher = [&](int id) {
        for (int i = 0; i < 10; ++i) {
            list.push_front(id * 100 + i);
        }
    };
    auto popper = [&]() {
        for (int i = 0; i < 10; ++i) {
            int v;
            if (list.pop_front(v)) std::cout << "Popped " << v << std::cout
                << endl;
        }
    };

    std::vector<std::thread> threads;
    threads.emplace_back(pusher, 1);
    threads.emplace_back(pusher, 2);
    threads.emplace_back(popper);
    for (auto &t : threads) t.join();
    std::cout << "Final size: " << list.size() << std::endl;
}

int main() {
    linkedListTest();
    return 0;
}

```

Explanation: The linked list is implemented with coarse-grained locking, using one mutex to guard the entire structure. Each operation (`push_front`, `pop_front`, `size`, and `empty`) locks the list, ensuring safe pointer updates and consistent traversal during concurrent modifications.

Analysis: This design is reliable and easy to understand, making it ideal for demonstrating basic synchronization concepts. However, scalability is limited since only one thread can modify or read at a time. In real-world systems, fine-grained or lock-free approaches would improve performance but require complex memory management to handle concurrency safely. For the purposes of this lab, the coarse lock approach provides a clear foundation for thread-safe list operations.

Screenshot: Include a screenshot of compiling and running `thread_safe_linked_list.cpp`.

```
● @camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ g++ -o thread_safe_linked_list thread_safe_linked_list.cpp -pthread
  -std=c++11
● @camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $ ./thread_safe_linked_list
Exercise: Thread-Safe Linked List
Popped 209
Popped 208
Popped 207
Popped 206
Popped 205
Popped 204
Popped 203
Popped 202
Popped 201
Popped 200
Final size: 10
○ @camcleve3 → /workspaces/cs375-fa25-mod4-as2-lab-thread (main) $
```

Figure 5: Compilation and execution of thread_safe_linked_list.cpp